Money Mirror API Documentation

Base URL: `https://localhost:7048`
Version: v1
Last Updated: February 2026

Authentication & Authorization

All authenticated endpoints require a JWT token in the `Authorization` header:

```
Authorization: Bearer <your_jwt_token>
```

Token Types:

- **Parent JWT:** Used by parents to access parent-only endpoints
- **Child JWT:** Used by children to access child-only endpoints
- **None/Public:** No authentication required

Token Expiration:

- Access Token: 15 minutes
- Refresh Token: 7 days

Parent APIs

Parent Authentication Endpoints

1. Register New Parent Account

Creates a new parent account and sends email confirmation.

- **Method:** `POST`
- **Path:** `/api/auth/register`
- **Authentication:** None
- **Description:** Registers a new parent account with email/password and sends confirmation email.

Request Body:

```
{
  "email": "john.smith@email.com",           // required, valid email format
```

```
  "password": "MyP@ssw0rd!",                  // required, min 8 chars, must
contain uppercase, lowercase, digit, special char
  "confirmPassword": "MyP@ssw0rd!",          // required, must match password
  "firstName": "John",                        // required, max 100 chars,
letters/spaces/hyphens/apostrophes only
  "lastName": "Smith",                        // required, max 100 chars,
letters/spaces/hyphens/apostrophes only
  "phoneNumber": "+1-234-567-8900"           // optional, max 20 chars
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Registration successful! Please check your email to confirm
your account.",
  "data": {
    "parentId": 1
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Email already registered, validation failed

```
{
  "success": false,
  "message": "Email is already registered",
  "data": null,
  "errors": null
}
```

When to call:

- When user clicks "Sign Up" on the registration screen
- Before login - new parents must register first

---

2. Confirm Email Address

Confirms a parent's email using the token from confirmation email.

- **Method:** `GET`
- **Path:** `/api/auth/confirm-email`
- **Authentication:** None
- **Description:** Activates parent account after email verification.

Query Parameters:

```
?email=john.smith@email.com&token=550e8400-e29b-41d4-a716-446655440000
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Email confirmed successfully! You can now log in.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Invalid token, expired token, email not found

```
{
  "success": false,
  "message": "Confirmation link has expired. Please request a new one.",
  "data": null,
  "errors": null
}
```

When to call:

- When parent clicks confirmation link in their email
- Usually handled automatically by email link redirect

---

3. Login Parent

Authenticates parent and returns JWT tokens.

- **Method:** `POST`
- **Path:** `/api/auth/login`
- **Authentication:** None
- **Description:** Logs in parent with email/password, returns access + refresh tokens.

Request Body:

```
{
  "email": "john.smith@email.com",          // required, valid email format
  "password": "MyP@ssw0rd!"                 // required
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Login successful",
```

```
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "550e8400-e29b-41d4-a716-446655440000",
    "accessTokenExpiration": "2026-02-05T12:30:00Z",
    "refreshTokenExpiration": "2026-02-12T12:15:00Z",
    "parentId": 1,
    "email": "john.smith@email.com",
    "fullName": "John Smith"
  },
  "errors": null
}
```

Common Errors:

- **401 Unauthorized:** Invalid credentials, email not confirmed, account deleted

```
{
  "success": false,
  "message": "Invalid email or password",
  "data": null,
  "errors": null
}
```

When to call:

- When parent enters credentials on login screen
- Store both tokens securely (refresh token in secure storage, access token in memory)

---

4. Refresh Access Token

Gets new access token using refresh token.

- **Method:** `POST`
- **Path:** `/api/auth/refresh-token`
- **Authentication:** None (but requires valid refresh token)
- **Description:** Generates new JWT tokens when access token expires.

Request Body:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  // required, can
be expired
  "refreshToken": "550e8400-e29b-41d4-a716-446655440000"       // required,
must be valid
}
```

Response - 200 OK:

```
{
```

```
  "success": true,
  "message": "Token refreshed successfully",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "new-refresh-token-here",
    "accessTokenExpiration": "2026-02-05T13:00:00Z",
    "refreshTokenExpiration": "2026-02-12T12:45:00Z",
    "parentId": 1,
    "email": "john.smith@email.com",
    "fullName": "John Smith"
  },
  "errors": null
}
```

Common Errors:

- **401 Unauthorized:** Invalid or expired refresh token

```
{
  "success": false,
  "message": "Invalid or expired refresh token. Please log in again.",
  "data": null,
  "errors": null
}
```

When to call:

- Automatically when access token expires (every 15 minutes)
- If refresh fails, redirect to login screen

---

5. Logout Parent

Revokes parent's refresh token (logout).

- **Method:** `POST`
- **Path:** `/api/auth/logout`
- **Authentication:** Parent JWT
- **Description:** Invalidates refresh token to prevent further token refresh.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Logged out successfully",
  "data": null,
```

```
  "errors": null
}
```

Common Errors:

- **401 Unauthorized:** Invalid JWT token
- **400 Bad Request:** Logout failed

When to call:

- When parent clicks "Logout" button
- Clear stored tokens from app after successful logout

---

6. Forgot Password

Sends password reset email.

- **Method:** `POST`
- **Path:** `/api/auth/forgot-password`
- **Authentication:** None
- **Description:** Generates reset token and emails reset link to parent.

Request Body:

```
{
  "email": "john.smith@email.com"          // required, valid email format
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "If that email address is registered, you will receive a
password reset link shortly.",
  "data": null,
  "errors": null
}
```

Note: Always returns success for security (prevents email enumeration)

When to call:

- When parent clicks "Forgot Password" on login screen

---

7. Reset Password

Resets password using token from email.

- **Method:** `POST`
- **Path:** `/api/auth/reset-password`
- **Authentication:** None
- **Description:** Changes parent's password using reset token.

Request Body:

```
{
  "email": "john.smith@email.com",                    // required
  "token": "550e8400-e29b-41d4-a716-446655440000",    // required, from reset
email
  "newPassword": "NewP@ssw0rd!",                       // required, same
validation as registration
  "confirmNewPassword": "NewP@ssw0rd!"                 // required, must
match newPassword
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Password has been reset successfully. Please log in with your
new password.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Invalid or expired token, validation failed

When to call:

- When parent submits new password after clicking reset link in email

---

8. Resend Confirmation Email

Sends new email confirmation link.

- **Method:** `POST`
- **Path:** `/api/auth/resend-confirmation`
- **Authentication:** None
- **Description:** Generates new confirmation token and sends new email.

Request Body:

```
{
  "email": "john.smith@email.com"          // required
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "A new confirmation link has been sent to your email address.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Email already confirmed, email not found

When to call:

- When parent didn't receive confirmation email or it expired

---

Parent Profile Management

9. Get My Profile

Gets parent's profile information.

- **Method:** `GET`
- **Path:** `/api/auth/my-profile`
- **Authentication:** Parent JWT
- **Description:** Returns parent's current profile data for viewing/editing.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Welcome, John!",
  "data": {
    "parentID": 1,
    "email": "john.smith@email.com",
    "firstName": "John",
```

```
    "lastName": "Smith",
    "phoneNumber": "+1-234-567-8900",
    "createdAt": "2026-01-15T10:30:00Z",
    "isEmailConfirmed": true,
    "totalChildren": 2
  },
  "errors": null
}
```

When to call:

- When parent navigates to "My Profile" screen

---

10. Update Profile

Updates parent's name and phone number.

- **Method:** `PUT`
- **Path:** `/api/auth/profile`
- **Authentication:** Parent JWT
- **Description:** Updates parent's first name, last name, and phone number.

Request Body:

```
{
  "firstName": "John",                      // required, same validation as
registration
  "lastName": "Smith",                      // required
  "phoneNumber": "+1-234-567-8900"         // optional
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": null,
  "errors": null
}
```

Note: Email changes handled separately (see Change Email endpoint)

When to call:

- When parent saves changes on "Edit Profile" screen

---

11. Change Email

Initiates email change process.

- **Method:** `POST`
- **Path:** `/api/auth/change-email`
- **Authentication:** Parent JWT
- **Description:** Sends verification email to new address, requires current password.

Request Body:

```
{
  "newEmail": "newemail@example.com",       // required, valid email format, max 255 chars
  "currentPassword": "MyP@ssw0rd!"          // required for security
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "A confirmation link has been sent to your new email address. Please verify it to complete the change.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** New email already registered, incorrect password

When to call:

- When parent submits new email on "Change Email" screen

---

12. Confirm Email Change

Confirms email change using token.

- **Method:** `GET`
- **Path:** `/api/auth/confirm-email-change`
- **Authentication:** None
- **Description:** Completes email change after verification.

Query Parameters:

```
?oldEmail=john.smith@email.com&newEmail=newemail@example.com&token=550e8400-
e29b-41d4-a716-446655440000
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Email changed successfully! Please log in again with your new
email address.",
  "data": null,
  "errors": null
}
```

Note: All refresh tokens are revoked - parent must log in again

When to call:

- When parent clicks confirmation link in new email address
- After success, redirect to login screen

---

13. Delete Parent Account

Soft deletes parent account with 30-day recovery.

- **Method:** `DELETE`
- **Path:** `/api/auth/delete-parent-account`
- **Authentication:** Parent JWT
- **Description:** Marks account as deleted, preserves children's data.

Request Body:

```
"MyP@ssw0rd!"          // required - send password as raw string in body
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Account scheduled for deletion. You have until 2026-03-07 to
recover your account. Your children's learning data has been preserved.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Incorrect password

When to call:

- When parent confirms account deletion on settings screen

---

14. Recover Deleted Account

Restores soft-deleted account within grace period.

- **Method:** `POST`
- **Path:** `/api/auth/recover-account`
- **Authentication:** None
- **Description:** Recovers account deleted within last 30 days.

Request Body:

```
{
  "email": "john.smith@email.com",          // required
  "password": "MyP@ssw0rd!"                 // required
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Account recovered successfully! Please log in and re-link your
children.",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Account not deleted, grace period expired, incorrect password

When to call:

- When parent tries to recover deleted account (link from deletion confirmation)

Parent Dashboard & Overview

15. Get My Dashboard

Gets parent's main dashboard with all children.

- **Method:** `GET`
- **Path:** `/api/auth/my-dashboard`
- **Authentication:** Parent JWT
- **Description:** Returns welcome message and quick cards for all linked children.

Request Body:

```
// No body required
```

Response - 200 OK:


```
{
  "success": true,
  "message": "Hello, John! 👋",
  "data": {
    "firstName": "John",
    "totalChildren": 2,
    "children": [
      {
        "childID": 1,
        "firstName": "Emma",
        "currentBalance": 125.50,
        "age": 8,
        "avatarUrl": null
      },
      {
        "childID": 2,
        "firstName": "Noah",
        "currentBalance": 50.00,
        "age": 6,
        "avatarUrl": null
      }
    ]
  },
  "errors": null
}
```


When to call:

- When parent logs in successfully
- When parent navigates to main dashboard/home screen
- Use children array to display child selection buttons at top

---

16. Get Child Detailed Card

Gets detailed view for specific child.

- **Method:** `GET`
- **Path:** `/api/auth/child/{childId}/card`

- **Authentication:** Parent JWT
- **Description:** Returns child's balance, stats, and allowance info in one call.

Path Parameters:

```
{childId} = 1          // required, child's ID
```

Response - 200 OK:


```
{
  "success": true,
  "message": "Emma's Overview",
  "data": {
    "childID": 1,
    "firstName": "Emma",
    "lastName": "Smith",
    "age": 8,
    "gender": "Girl",
    "currentBalance": 125.50,
    "quickStats": {
      "totalSpentThisMonth": 45.00,
      "expensesCountThisMonth": 8,
      "activeGoalsCount": 2,
      "personalityTypeName": "Goal-Oriented Planner"
    },
    "allowanceInfo": {
      "type": "Weekly",
      "amount": 50.00,
      "nextPaymentDate": "2026-02-08T00:00:00Z",
      "isActive": true
    }
  },
  "errors": null
}
```


Note: `allowanceInfo` is null if no allowance is set up

Common Errors:

- **400 Bad Request:** Not authorized to view this child

When to call:

- When parent clicks on a child's button from dashboard
- Shows the child's section with balance and quick stats

Parent Child Management

17. Complete Initial Profiling (Create New Child)

Creates new child with questionnaire and personality assignment.

- **Method:** `POST`
- **Path:** `/api/children/complete-initial-profiling`
- **Authentication:** Parent JWT
- **Description:** Creates child account, saves questionnaire, calls Python AI for personality, generates login code.

Request Body:

```
{
  "childFirstName": "Emma",                              // required, max 100
chars
  "childLastName": "Smith",                              // required, max 100
chars
  "dob": "2017-03-15T00:00:00Z",                        // required, must be
valid child age (0-18)
  "gender": "Girl",                                      // optional, "Boy" or
"Girl" or null
  "hasAllowance": "Yes",                                 // required enum: Yes,
No
  "spendingPace": "Spends_Gradually",                    // required enum:
Spends_Right_Away, Spends_Gradually, Saves_Part_Of_It
  "spendingCategories": [                                // required array, 1-4
items
    "Food_And_Drinks",
    "Entertainment"
  ],
  "outOfMoneyBehavior": "Postpone_Purchases",            // required enum:
Ask_For_More, Stop_Spending, Postpone_Purchases
  "triesToSave": "Yes",                                  // required enum: Yes,
No
  "moneyMindset": "Balances_Spending_And_Saving",        // required enum:
Enjoys_Spending_Immediately, Balances_Spending_And_Saving,
Saves_For_The_Future
  "feelingAfterSpending": "Happy",                       // required enum:
Happy, Regretful, Neutral
  "feelingWhenSavingGrows": "Motivated",                 // required enum:
Motivated, Proud, Doesnt_Matter_Much
  "reactionTo100": "Spend_Part_Save_Part"                // required enum:
Spend_All_Now, Spend_Part_Save_Part, Save_All_For_Future
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Questionnaire completed successfully! Your child's login code
is ready.",
  "data": {
    "childID": 1,
    "childFirstName": "Emma",
    "age": 8,
    "gender": "Girl",
```

```
    "childLoginCode": "ABC123",
    "isPersonalityFinalized": true,         // true if Python AI succeeded,
false if fallback used
    "personalityProfile": {
      "typeID": 3,
      "parentName": "Goal-Oriented Planner",
      "childName": "Dream Builder",
      "description": "...",
      "funFacts": "You're great at planning your purchases! 🎯"
    }
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Validation failed

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Child must be between 0 and 18 years old",
    "Please select at least one spending category"
  ]
}
```

When to call:

- After parent completes "Add New Child" questionnaire flow
- **Important:** Save the `childLoginCode` - parent needs to give this to child for their login

---

18. Add Existing Child

Links existing child to parent account using login code.

- **Method:** `POST`
- **Path:** `/api/children/add-existing`
- **Authentication:** Parent JWT
- **Description:** Adds child created by another parent (shared custody scenario).

Request Body:

```
{
  "code": "ABC123"          // required, 6 characters, uppercase letters and
numbers
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Child Emma added successfully!",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Invalid code, child already linked to this parent

When to call:

- When parent enters a child's login code on "Add Existing Child" screen
- Typically used for divorced/separated parents sharing custody

---

19. Get My Children

Lists all children linked to parent.

- **Method:** `GET`
- **Path:** `/api/children/my-children`
- **Authentication:** Parent JWT
- **Description:** Returns list of all children for "Manage Children" tab.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Found 2 child(ren)",
  "data": [
    {
      "childID": 1,
      "childName": "Emma Smith",
      "dob": "2017-03-15T00:00:00Z",
      "age": 8,
      "gender": "Girl",
      "loginCode": "ABC123",
      "createdAt": "2026-01-15T14:30:00Z",
      "isPersonalityFinalized": true,
      "personalityTypeName": "Goal-Oriented Planner"
    },
```

```
    {
      "childID": 2,
      "childName": "Noah Smith",
      "dob": "2019-08-22T00:00:00Z",
      "age": 6,
      "gender": "Boy",
      "loginCode": "XYZ789",
      "createdAt": "2026-01-20T10:00:00Z",
      "isPersonalityFinalized": false,
      "personalityTypeName": "Pending Analysis"
    }
  ],
  "errors": null
}
```

When to call:

- When parent navigates to "Manage Children" tab/screen
- Use for displaying list of children with their login codes

---

20. Update Child Info

Updates child's name and date of birth.

- **Method:** `PUT`
- **Path:** `/api/children/{childId}`
- **Authentication:** Parent JWT
- **Description:** Updates basic child information, recalculates age automatically.

Path Parameters:

```
{childId} = 1          // required
```

Request Body:

```
{
  "firstName": "Emma",                      // required, same validation as
create
  "lastName": "Smith",                      // required
  "dateOfBirth": "2017-03-15T00:00:00Z"    // required, age must be 0-18
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Child Emma updated successfully!",
  "data": {
    "childID": 1,
```

```
    "firstName": "Emma",
    "lastName": "Smith",
    "dateOfBirth": "2017-03-15T00:00:00Z",
    "age": 8,
    "ageGroup": "6-8 years old"
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Not authorized to update this child, validation failed

When to call:

- When parent saves changes on "Edit Child Info" screen

---

21. Regenerate Child Login Code

Generates new login code for child.

- **Method:** `POST`
- **Path:** `/api/children/{childId}/regenerate-code`
- **Authentication:** Parent JWT
- **Description:** Creates new login code, invalidates old one, forces child to log in again.

Path Parameters:

```
{childId} = 1          // required
```

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "New login code generated for Emma Smith. The old code is no
longer valid.",
  "data": {
    "newLoginCode": "DEF456",
    "childName": "Emma Smith"
  },
  "errors": null
}
```

Note: Old code becomes invalid immediately, child's refresh tokens are revoked

When to call:

- When parent clicks "Regenerate Code" on child management screen
- If child's code is lost or compromised

---

22. Delete Child Account

Permanently deletes child and all their data.

- **Method:** `DELETE`
- **Path:** `/api/children/{childId}`
- **Authentication:** Parent JWT
- **Description:** Hard deletes child - cannot be undone. Removes all expenses, goals, transactions.

Path Parameters:

```
{childId} = 1          // required
```

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Child Emma Smith and all their data have been permanently
deleted",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Not authorized to delete this child

When to call:

- When parent confirms child deletion on settings/manage children screen
- **Warning:** Show confirmation dialog - this is permanent!

Child APIs

Child Authentication Endpoints

23. Child Login with Code

Authenticates child using their login code.

- **Method:** `POST`
- **Path:** `/api/children/login-with-code`
- **Authentication:** None
- **Description:** Logs in child with 6-character code, returns JWT tokens.

Request Body:

```
{
  "code": "ABC123"          // required, 6 characters, uppercase letters and
numbers
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Welcome back, Emma!",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "550e8400-e29b-41d4-a716-446655440000",
    "accessTokenExpiration": "2026-02-05T12:30:00Z",
    "refreshTokenExpiration": "2026-02-12T12:15:00Z",
    "childId": 1,
    "childFirstName": "Emma",
    "age": 8,
    "isPersonalityFinalized": true,
    "personalityProfile": {
      "typeID": 3,
      "parentName": "Goal-Oriented Planner",
      "childName": "Dream Builder",
      "description": "...",
      "funFacts": "You're great at planning! 🎯"
    }
  },
  "errors": null
}
```

Common Errors:

- **401 Unauthorized:** Invalid or expired code

```
{
  "success": false,
  "message": "Invalid or expired code",
  "data": null,
  "errors": null
```

```
}
```

When to call:

- When child enters their login code on child login screen
- Store tokens securely like parent login

---

24. Child Refresh Token

Gets new access token for child.

- **Method:** `POST`
- **Path:** `/api/children/refresh-token`
- **Authentication:** None (requires valid refresh token)
- **Description:** Refreshes child's JWT tokens.

Request Body:

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "550e8400-e29b-41d4-a716-446655440000"
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Tokens refreshed successfully",
  "data": {
    "accessToken": "new-access-token",
    "refreshToken": "new-refresh-token",
    "accessTokenExpiration": "2026-02-05T13:00:00Z",
    "refreshTokenExpiration": "2026-02-12T12:45:00Z",
    "childId": 1,
    "childFirstName": "Emma",
    "age": 8,
    "isPersonalityFinalized": true,
    "personalityProfile": { /* ... */ }
  },
  "errors": null
}
```

Common Errors:

- **401 Unauthorized:** Invalid or expired refresh token

When to call:

- Automatically when child's access token expires (every 15 minutes)

---

25. Child Logout

Logs out child by revoking refresh token.

- **Method:** `POST`
- **Path:** `/api/children/logout`
- **Authentication:** Child JWT
- **Description:** Invalidates child's refresh token.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Logged out successfully. See you next time!",
  "data": null,
  "errors": null
}
```

When to call:

- When child clicks logout button
- Clear stored tokens after successful logout

Child Dashboard & Profile

26. Get My Profile (Child)

Gets child's profile information.

- **Method:** `GET`
- **Path:** `/api/children/my-profile`
- **Authentication:** Child JWT
- **Description:** Returns child's profile data for "My Profile" screen.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Welcome, Emma!",
  "data": {
    "childID": 1,
    "firstName": "Emma",
    "lastName": "Smith",
    "age": 8,
    "gender": "Girl",
    "currentBalance": 125.50,
    "personalityInfo": {
      "childName": "Dream Builder",
      "funFacts": "You're great at planning your purchases! 🎯",
      "isFinalized": true
    },
    "avatarUrl": null
  },
  "errors": null
}
```

When to call:

- When child navigates to "My Profile" screen

---

27. Get My Dashboard (Child)

Gets child's main dashboard.

- **Method:** `GET`
- **Path:** `/api/children/my-dashboard`
- **Authentication:** Child JWT
- **Description:** Returns child's home screen data with balance and stats.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Hello, Emma! 👋",
  "data": {
    "firstName": "Emma",
    "currentBalance": 125.50,
    "avatarUrl": null,
    "personalityName": "Dream Builder",
```

```
    "funFacts": "You're great at planning! 🎯",
    "unloggedExpensesCount": 0,
    "activeGoalsCount": 2
  },
  "errors": null
}
```

When to call:

- When child logs in successfully
- When child navigates to home/dashboard screen

Child Character Selection

28. Get Available Characters

Lists all space-themed characters child can choose.

- **Method:** `GET`
- **Path:** `/api/character/available`
- **Authentication:** None
- **Description:** Returns all characters for selection screen.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Found 4 available characters",
  "data": [
    {
      "characterID": 1,
      "name": "Nova",
      "description": "A friendly space explorer who loves adventures!",
      "imageUrl": "/images/characters/nova/default.png"
    },
    {
      "characterID": 2,
      "name": "Luna",
      "description": "A curious moon traveler with big dreams!",
      "imageUrl": "/images/characters/luna/default.png"
    }
    // ... more characters
  ],
  "errors": null
}
```

When to call:

- When child first logs in (if no character selected yet)
- When child navigates to character selection screen

---

29. Select Character

Assigns chosen character to child.

- **Method:** `PUT`
- **Path:** `/api/character/select`
- **Authentication:** Child JWT
- **Description:** Sets child's selected character.

Request Body:

```
{
  "characterID": 1          // required
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "You chose Nova! 🚀",
  "data": {
    "characterID": 1,
    "characterName": "Nova",
    "message": "You chose Nova! 🚀"
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Character not found

When to call:

- When child selects a character for the first time
- When child changes their character

---

30. Get Character State for Screen

Gets character image for specific screen context.

- **Method:** `POST`
- **Path:** `/api/character/state`
- **Authentication:** Child JWT
- **Description:** Returns appropriate character image and message for current screen.

Request Body:

```
{
  "screenContext": "Dashboard"    // required: "Dashboard", "Expenses",
"Savings", "Goals", "Profile"
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Character state retrieved successfully",
  "data": {
    "characterName": "Nova",
    "imageUrl": "/images/characters/nova/dashboard.png",
    "message": "Let's see how you're doing today! 🚀"
  },
  "errors": null
}
```

When to call:

- When child navigates to a new screen
- To display appropriate character image/message for context

---

31. Get Profile Picture

Gets child's character image for profile.

- **Method:** `GET`
- **Path:** `/api/character/profile-picture`
- **Authentication:** Child JWT
- **Description:** Returns character default image for child's profile picture.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Profile picture retrieved successfully",
  "data": "/images/characters/nova/default.png",
  "errors": null
}
```

When to call:

- When displaying child's profile picture
- When showing child's avatar in lists

---

32. Get Child Profile Picture (Parent View)

Gets child's character image (parent can view).

- **Method:** `GET`
- **Path:** `/api/character/{childId}/profile-picture`
- **Authentication:** Parent JWT
- **Description:** Returns character image for parent viewing child's profile.

Path Parameters:

```
{childId} = 1          // required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Profile picture retrieved successfully",
  "data": "/images/characters/nova/default.png",
  "errors": null
}
```

When to call:

- When parent views child's profile/card
- When displaying child's avatar in parent's child list

Allowance & Balance APIs

Parent Allowance Management

33. Update Recurring Allowance

Creates or updates child's allowance schedule.

- **Method:** `PUT`
- **Path:** `/api/allowance/{childId}`
- **Authentication:** Parent JWT
- **Description:** Sets recurring allowance (daily/weekly/monthly) or updates existing one.

Path Parameters:

```
{childId} = 1          // required
```

Request Body:

```
{
  "type": "Weekly",                        // required: "Daily", "Weekly", or
"Monthly"
  "amount": 50.00,                         // required, must be > 0 and <=
10000
  "dailyHour": null,                       // required if type="Daily", 0-23
  "weeklyDay": "Saturday",                 // required if type="Weekly":
"Saturday"-"Friday"
  "monthlyDay": null,                      // required if type="Monthly", 1-
31
  "isActive": true                         // optional, default true
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Allowance settings updated successfully",
  "data": null,
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Not authorized, validation failed

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Weekly day is required for weekly allowances"
  ]
}
```

When to call:

- When parent sets up allowance for first time

- When parent changes existing allowance settings
- **Note:** If allowance exists, it will be updated (not duplicated)

---

34. Get Allowance Settings

Gets child's current allowance configuration.

- **Method:** `GET`
- **Path:** `/api/allowance/{childId}`
- **Authentication:** Parent JWT
- **Description:** Returns active recurring allowance details or null if not set up.

Path Parameters:

```
{childId} = 1          // required
```

Response - 200 OK (Allowance exists):

```
{
  "success": true,
  "message": "Allowance settings retrieved successfully",
  "data": {
    "allowanceId": 1,
    "type": "Weekly",
    "amount": 50.00,
    "isActive": true,
    "dailyHour": null,
    "weeklyDay": "Saturday",
    "monthlyDay": null,
    "lastCreditedDate": "2026-02-01T00:00:00Z",
    "setDate": "2026-01-15T10:00:00Z"
  },
  "errors": null
}
```

Response - 200 OK (No allowance):

```
{
  "success": true,
  "message": "No allowance is currently set up for this child",
  "data": null,
  "errors": null
}
```

When to call:

- When parent opens "Manage Allowance" screen
- To pre-fill form with current settings for editing

---

35. Give Bonus

Gives one-time bonus to child.

- **Method:** `POST`
- **Path:** `/api/allowance/{childId}/bonus`
- **Authentication:** Parent JWT
- **Description:** Immediately credits bonus amount to child's balance.

Path Parameters:

```
{childId} = 1          // required
```

Request Body:

```
{
  "amount": 100.00,                                    // required, > 0 and
<= 50000
  "reason": "Excellent report card!"                   // required, min 3
chars, max 500 chars
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Bonus of 100.00 given successfully!",
  "data": {
    "newBalance": 225.50
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Not authorized, validation failed

When to call:

- When parent gives reward/bonus to child
- Balance is updated immediately

---

36. Get Child Balance (Parent View)

Gets child's current balance.

- **Method:** `GET`
- **Path:** `/api/allowance/{childId}/balance`
- **Authentication:** Parent JWT
- **Description:** Returns child's current account balance.

Path Parameters:

```
{childId} = 1          // required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Balance retrieved successfully",
  "data": {
    "currentBalance": 125.50,
    "childId": 1,
    "childName": "Emma Smith"
  },
  "errors": null
}
```

When to call:

- When parent views child's balance card
- Typically included in child detailed card (endpoint #16)

---

37. Get Transaction History (Parent View)

Gets child's transaction history with filters.

- **Method:** `GET`
- **Path:** `/api/allowance/{childId}/transactions`
- **Authentication:** Parent JWT
- **Description:** Returns filtered list of allowances, bonuses, and expenses.

Path Parameters:

```
{childId} = 1          // required
```

Query Parameters:

```
?startDate=2026-01-01T00:00:00Z          // optional
&endDate=2026-01-31T23:59:59Z            // optional
&type=All                                 // optional: "All",
"AllowanceCredit", "BonusCredit", "Expense"
```

Response - 200 OK:


```
{
  "success": true,
  "message": "Found 15 transaction(s)",
  "data": {
    "transactions": [
      {
        "transactionId": 45,
        "type": "AllowanceCredit",
        "amount": 50.00,
        "balanceAfter": 225.50,
        "description": "Weekly allowance credited",
        "transactionDate": "2026-02-01T00:00:00Z"
      },
      {
        "transactionId": 44,
        "type": "BonusCredit",
        "amount": 100.00,
        "balanceAfter": 175.50,
        "description": "Excellent report card!",
        "transactionDate": "2026-01-28T15:30:00Z"
      },
      {
        "transactionId": 43,
        "type": "Expense",
        "amount": 15.50,
        "balanceAfter": 75.50,
        "description": "Spent on Ice Cream",
        "transactionDate": "2026-01-25T14:20:00Z"
      }
    ],
    "totalCredits": 450.00,
    "totalCount": 15
  },
  "errors": null
}
```


When to call:

- When parent views "Transaction History" screen
- To show allowance/bonus payment history

Child Allowance Views

38. Get My Balance (Child)

Gets child's own balance.

- **Method:** `GET`

- **Path:** `/api/allowance/my-balance`
- **Authentication:** Child JWT
- **Description:** Returns child's current balance with friendly message.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "You have 125.50 in your account! 💰",
  "data": {
    "currentBalance": 125.50,
    "childId": 1,
    "childName": "Emma Smith"
  },
  "errors": null
}
```

When to call:

- When child opens dashboard/home screen
- To display balance prominently

---

39. Get My Transactions (Child)

Gets child's transaction history.

- **Method:** `GET`
- **Path:** `/api/allowance/my-transactions`
- **Authentication:** Child JWT
- **Description:** Returns child's allowance and bonus history (simplified view).

Query Parameters:

```
?startDate=2026-01-01T00:00:00Z          // optional
```

Response - 200 OK:

```
{
  "success": true,
  "message": "You have 8 transaction(s)",
  "data": {
    "transactions": [
      {
```

```
        "transactionId": 45,
        "type": "AllowanceCredit",
        "amount": 50.00,
        "balanceAfter": 225.50,
        "description": "Weekly allowance credited",
        "transactionDate": "2026-02-01T00:00:00Z"
      }
      // ... more transactions
    ],
    "totalCredits": 300.00,
    "totalCount": 8
  },
  "errors": null
}
```

When to call:

- When child views "My Money" or transaction history screen

---

40. Get My Allowance Info (Child)

Gets child-friendly allowance information.

- **Method:** `GET`
- **Path:** `/api/allowance/my-allowance`
- **Authentication:** Child JWT
- **Description:** Returns when child will get paid next.

Request Body:

```
// No body required
```

Response - 200 OK (Has allowance):

```
{
  "success": true,
  "message": "You get 50.00 every Saturday! 🎉",
  "data": {
    "message": "You get 50.00 every Saturday! 🎉",
    "nextPaymentDate": "2026-02-08T00:00:00Z",
    "isActive": true,
    "amount": 50.00
  },
  "errors": null
}
```

Response - 200 OK (No allowance):

```
{
```

```
  "success": true,
  "message": "You don't have an allowance set up yet. Ask your parent!",
  "data": {
    "message": "You don't have an allowance set up yet. Ask your parent!",
    "nextPaymentDate": null,
    "isActive": false,
    "amount": 0
  },
  "errors": null
}
```

When to call:

- When child views allowance/money information screen
- To show when they'll get paid next

Expense APIs

Child Expense Logging

41. Log Expense

Records a new expense for child.

- **Method:** `POST`
- **Path:** `/api/expense/log`
- **Authentication:** Child JWT
- **Description:** Logs expense, decreases balance, creates transaction record.

Request Body:

```
{
  "itemName": "Ice Cream",                  // required, min 2 chars, max 200
chars
  "categoryID": 2,                          // required, must be valid
category (see Get Categories)
  "moodID": 1,                              // required, must be valid mood
(see Get Moods)
  "moneyAmount": 15.50,                     // required, > 0 and <= 10000 and
<= currentBalance
  "note": "Shared with my friend"          // optional, max 500 chars
}
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Expense logged! You spent 15.50. Your new balance is 110.00 💰",
```

```
  "data": {
    "expenseID": 25,
    "itemName": "Ice Cream",
    "categoryName": "Food & Snacks",
    "moodDescription": "Happy",
    "amount": 15.50,
    "logDate": "2026-02-05T14:30:00Z",
    "note": "Shared with my friend"
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Insufficient balance, invalid category/mood

```
{
  "success": false,
  "message": "Insufficient balance. You have 10.00 but tried to spend 15.50",
  "data": null,
  "errors": null
}
```

When to call:

- When child logs a purchase on "Add Expense" screen
- Balance is decreased immediately

---

42. Get My Expenses (Child)

Gets child's expense history.

- **Method:** `GET`
- **Path:** `/api/expense/my-expenses`
- **Authentication:** Child JWT
- **Description:** Returns filtered list of child's expenses.

Query Parameters:

```
?startDate=2026-01-01T00:00:00Z          // optional
&endDate=2026-01-31T23:59:59Z            // optional
&categoryId=2                             // optional, filter by category
&moodId=1                                 // optional, filter by mood
```

Response - 200 OK:

```
{
  "success": true,
  "message": "You have 8 expense(s). Total spent: 127.50",
```

```
  "data": {
    "expenses": [
      {
        "expenseID": 25,
        "itemName": "Ice Cream",
        "categoryName": "Food & Snacks",
        "moodDescription": "Happy",
        "amount": 15.50,
        "logDate": "2026-02-05T14:30:00Z",
        "note": "Shared with my friend"
      }
      // ... more expenses
    ],
    "totalCount": 8,
    "totalSpent": 127.50
  },
  "errors": null
}
```

When to call:

- When child views "My Expenses" or "Spending History" screen
- Use filters to show specific time periods or categories

Parent Expense Viewing

43. Get Child Expenses (Parent View)

Gets child's expenses (parent can view and filter).

- **Method:** `GET`
- **Path:** `/api/expense/{childId}/expenses`
- **Authentication:** Parent JWT
- **Description:** Returns child's expense history with parent-level filtering.

Path Parameters:

```
{childId} = 1          // required
```

Query Parameters:

```
?startDate=2026-01-01T00:00:00Z          // optional
&endDate=2026-01-31T23:59:59Z            // optional
&categoryId=2                             // optional
&moodId=1                                 // optional
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Found 15 expense(s). Total spent: 245.00",
  "data": {
    "expenses": [
      {
        "expenseID": 25,
        "itemName": "Ice Cream",
        "categoryName": "Food & Snacks",
        "moodDescription": "Happy",
        "amount": 15.50,
        "logDate": "2026-02-05T14:30:00Z",
        "note": "Shared with my friend"
      }
      // ... more expenses
    ],
    "totalCount": 15,
    "totalSpent": 245.00
  },
  "errors": null
}
```

Common Errors:

- **400 Bad Request:** Not authorized to view this child's expenses

When to call:

- When parent views child's "Expense History" screen
- To analyze spending patterns by category/mood

Expense Shared Endpoints

44. Get Categories

Lists all expense categories.

- **Method:** `GET`
- **Path:** `/api/expense/categories`
- **Authentication:** Parent JWT or Child JWT
- **Description:** Returns all available expense categories for dropdown.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Found 8 categories",
  "data": [
    {
      "categoryID": 1,
      "name": "Toys"
    },
    {
      "categoryID": 2,
      "name": "Food & Snacks"
    },
    {
      "categoryID": 3,
      "name": "Books"
    },
    {
      "categoryID": 4,
      "name": "Clothes"
    },
    {
      "categoryID": 5,
      "name": "Entertainment"
    },
    {
      "categoryID": 6,
      "name": "School Supplies"
    },
    {
      "categoryID": 7,
      "name": "Games"
    },
    {
      "categoryID": 8,
      "name": "Other"
    }
  ],
  "errors": null
}
```


When to call:

- When loading "Log Expense" screen (to populate category dropdown)
- Cache this list in app - categories rarely change

---

45. Get Moods

Lists all mood options.

- **Method:** `GET`
- **Path:** `/api/expense/moods`

- **Authentication:** Parent JWT or Child JWT
- **Description:** Returns all available moods for mood picker.

Request Body:

```
// No body required
```

Response - 200 OK:

```
{
  "success": true,
  "message": "Found 5 moods",
  "data": [
    {
      "moodID": 1,
      "description": "Happy"
    },
    {
      "moodID": 2,
      "description": "Sad"
    },
    {
      "moodID": 3,
      "description": "Neutral"
    },
    {
      "moodID": 4,
      "description": "Excited"
    },
    {
      "moodID": 5,
      "description": "Regretful"
    }
  ],
  "errors": null
}
```

When to call:

- When loading "Log Expense" screen (to populate mood picker)
- Cache this list in app - moods rarely change

Common Response Structure

All API responses follow this consistent structure:

```
{
  "success": true,                    // boolean: true if successful, false
if error
  "message": "Operation successful",  // string: human-readable message
```

```
  "data": { /* ... */ },              // object/array/null: actual response
data
  "errors": null                       // array of strings or null: validation
errors
}
```

Success Response Example:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "accessToken": "...",
    "parentId": 1
  },
  "errors": null
}
```

Error Response Example:

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Email is required",
    "Password must be at least 8 characters"
  ]
}
```

Error Handling

HTTP Status Codes

- **200 OK:** Success (check `success` field in body)
- **400 Bad Request:** Validation error, business logic error
- **401 Unauthorized:** Missing/invalid JWT token, invalid credentials
- **404 Not Found:** Endpoint doesn't exist
- **500 Internal Server Error:** Server error (rare - log and report)

Common Error Scenarios

1. Validation Errors (400)

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Email is required",
```

```
    "Password must be at least 8 characters"
  ]
}
```

Action: Show errors to user in form

2. Authentication Errors (401)

```
{
  "success": false,
  "message": "Invalid or expired refresh token. Please log in again.",
  "data": null,
  "errors": null
}
```

Action: Clear tokens, redirect to login

3. Authorization Errors (400)

```
{
  "success": false,
  "message": "You are not authorized to view this child",
  "data": null,
  "errors": null
}
```

Action: Show error message, prevent action

4. Business Logic Errors (400)

```
{
  "success": false,
  "message": "Insufficient balance. You have 10.00 but tried to spend 15.50",
  "data": null,
  "errors": null
}
```

Action: Show error message to user

Important Notes for Frontend Team

Authentication Flow

1. **Parent Login:** Use endpoint #3 → Store both tokens
2. **Auto-refresh:** When access token expires, use endpoint #4
3. **If refresh fails:** Clear tokens, redirect to login
4. **Logout:** Call endpoint #5, clear tokens locally

Child Login Flow

1. **Child Login:** Use endpoint #23 with code parent gave them
2. **Same refresh logic** as parent (endpoint #24)
3. **Logout:** Endpoint #25

Token Storage

- **Access Token:** Store in memory (short-lived, 15 min)
- **Refresh Token:** Store in secure storage (7 days)
- **Never store passwords**

Making Authenticated Requests

```
headers: {
  'Authorization': `Bearer ${accessToken}`,
  'Content-Type': 'application/json'
}
```

Handling Balance Updates

Balance-changing operations (log expense, give bonus, credit allowance) return new balance in response. Update local state immediately to avoid extra API calls.

Caching Recommendations

These rarely change - cache locally:

- Categories (endpoint #44)
- Moods (endpoint #45)
- Characters (endpoint #28)

These change often - refresh frequently:

- Balance
- Expenses
- Transactions

Dates and Timezones

All dates are in UTC (ISO 8601 format). Convert to local timezone in app when displaying to users.

Example: `"2026-02-05T14:30:00Z"` = UTC

Summary of User Flows

Parent Registration → First Child

1. Register (endpoint #1)
2. Confirm email (endpoint #2)
3. Login (endpoint #3)
4. Complete questionnaire for child (endpoint #17)
5. Save child's login code to give them

Parent Daily Use

1. Login (endpoint #3)
2. View dashboard (endpoint #15)
3. Click on child (endpoint #16)
4. Manage allowance (endpoints #33, #34, #35)
5. View expenses (endpoint #43)
6. View transactions (endpoint #37)

Child Daily Use

1. Login with code (endpoint #23)
2. View dashboard (endpoint #27)
3. Log expense (endpoint #41)
4. View balance (endpoint #38)
5. View expenses (endpoint #42)

End of API Documentation